

Sumário

1. [Introdução](#)
2. [Parte 1 — Revisão da Arquitetura](#)
3. [Parte 2 — Revisão de Segurança](#)
4. [Parte 3 — Melhoria de Código Gerado com IA](#)
5. [Parte 4 — Implementação de uma Camada de Web Scraping](#)
6. [Conclusão](#)
7. [Apêndice — Estrutura Final do Projeto](#)

Introdução

O **AgroVision AI** é uma aplicação que junta visão computacional (YOLOv8), persistência (SQLite) e IA generativa local (Ollama) num pipeline único. O sistema lê um stream de vídeo, detecta objetos relevantes, registra eventos e expõe um agente de linguagem natural que comenta o que está acontecendo na cena.

Nesta atividade, nosso grupo revisou criticamente o projeto sob quatro óticas: **arquitetura, segurança, qualidade do código auxiliado por IA e integração externa via web scraping**. A ideia central que guiou nosso trabalho foi a do enunciado: usar IA para gerar código não dispensa o crivo humano. Quase tudo do projeto inicial "funcionava", mas funcionar não é a mesma coisa que estar bem arquitetado, seguro e sustentável.

Este relatório registra o que **encontramos**, o que **decidimos mudar**, o que **decidimos deixar como está** e a **justificativa** de cada decisão.

Parte 1 — Revisão da Arquitetura

Começamos olhando para a estrutura do projeto perguntando se ela suportaria crescimento, manutenção e a chegada de novas funcionalidades (especialmente a camada de scraping da Parte 4). Nossa primeira impressão foi positiva: o projeto **já tinha camadas separadas, backend concentrando a lógica e banco isolado**. Porém, descobrimos um ponto crítico em `services/video_monitor.py`: o método `_process_frame` misturava inferência YOLO, regra de negócio e persistência. Esse foi o foco do nosso refator.

1.1 Padrão arquitetural adotado

Para deixar nossas decisões defensáveis, escolhemos uma **combinação de três padrões**:

- **Layered Architecture** como esqueleto geral (Presentation → API → Application → Domain → Infrastructure). Era o que já estava implícito; nós só formalizamos.
- **Hexagonal (Ports & Adapters)** para isolar as integrações externas. Definimos interfaces explícitas com `typing.Protocol` (`@runtime_checkable`) que cada adapter precisa cumprir.
- **Pipeline (Pipes & Filters)** para modelar o fluxo de visão computacional: `frame → Detector.detect → Detector.annotate → AlertGate.submit → EventStore.save`.

Consideramos Clean Architecture e DDD tático, mas descartamos: o projeto é pequeno demais para justificar tantas camadas e mappers.

PRESENTATION	templates/ + static/		
API (transport)	app.py		
APPLICATION	VideoMonitor + MonitoringAgent (functional core)		
DOMAIN (puro)	services/domain.py		
	value objects: Detection, AlertEvent		
	ports:	Detector, AlertGate, EventStore,	
		LlmClient, RateLimiter	
INFRASTRUCTURE	vision/	YoloDetector	→ Detector
	alerts/	AlertEngine	→ AlertGate
	persistence/	EventRepository	→ EventStore
	agent/	OllamaClient	→ LlmClient
	security/	...	
	external/	(Parte 4: scraping)	

1.2 Mapeamento das sete camadas

A atividade pede que verifiquemos a divisão entre sete camadas. Resumimos abaixo o estado de cada uma após nosso trabalho:

Camada	Onde está hoje	Mudou?
Frontend	<code>templates/</code> + <code>static/</code>	Não
Backend / API	<code>app.py</code> (composition root + rotas)	Apenas imports atualizados
Banco de dados	<code>services/persistence/event_repository.py</code>	Sim — API adaptada
Serviços internos	<code>services/</code> em subpacotes	Sim — reorganização
Camada de IA / Modelo	<code>services/vision/</code> + <code>services/agent/</code>	Sim — refator principal
Integração externa	<code>services/agent/ollama_client.py</code> + <code>services/external/</code>	Formalizada
Web scraping	<code>services/external/</code> (placeholder)	A criar na Parte 4

1.3 Frontend

Olhamos o `dashboard.js`, o `index.html` e o `dashboard.css`. Tudo que o cliente faz é polling de endpoints, montagem de DOM e gestão local do histórico do chat — **nenhuma regra de negócio**. Por isso decidimos **não tocar nessa camada**. Mexer só pra mostrar evolução seria mudança sem propósito; deixá-la intacta inclusive validou nosso refator de backend, porque o `dashboard.js` continuou funcionando sem ajuste depois de toda a reorganização.

1.4 Backend / API

O `app.py` já era **fino**: só roteamento e composição. Nenhuma rota implementava lógica própria; todas delegavam para serviços. Como a Parte 1 não pedia mudança aqui, mantivemos o desenho — só atualizamos os imports como consequência da reorganização dos subpacotes (de `from services.event_repository import ...` para `from services.persistence.event_repository import ...`, e assim por diante).

1.5 Banco de dados

Antes: o acesso ao banco já estava isolado em `EventRepository`. Confirmamos com `grep` que nenhum outro arquivo tocava SQL. Mas a API era pouco semântica: `save_event(id, label, conf, path)` recebia quatro argumentos posicionais soltos, e não havia um tipo de domínio representando "evento".

Depois: introduzimos a dataclass `AlertEvent` em `services/domain.py` e renomeamos os métodos do repositório:

```
class EventRepository:
    def init(self) → None: ...
    def save(self, event: AlertEvent) → None: ...
    def list_recent(self, limit: int = 50) → list[dict]: ...
```

Importante: `list_recent` continua devolvendo `list[dict]` porque esse é o **contrato público** consumido pelo `dashboard.js`. Mudar o formato quebraria a UI sem ganho real.

Por que melhorou: chamadas agora passam um objeto coeso em vez de argumentos soltos; o JSON em `/events` foi preservado; trocar SQLite por outro backend (por exemplo, PostgreSQL) vira só criar um novo adapter que cumpra o port `EventStore`.

1.6 Serviços internos

Antes: oito arquivos `.py` num único nível dentro de `services/`. Funcionava, mas a estrutura não comunicava as camadas — era preciso abrir cada arquivo para entender quem fazia o quê.

Depois: reorganizamos em **cinco subpacotes**:

```
services/
├─ domain.py          (value objects + ports – novo)
├─ config.py
├─ video_monitor.py   (orquestrador)
├─ vision/            yolo_detector.py, capture_store.py
├─ alerts/            alert_engine.py (novo)
├─ persistence/       event_repository.py
├─ agent/             monitoring_agent.py, ollama_client.py, schemas.py
└─ external/          (reservado para Parte 4)
```

Por que melhorou: a estrutura passou a se auto-documentar. Um novo desenvolvedor enxerga a arquitetura pelo nome das pastas; saber onde uma nova feature entra fica óbvio.

1.7 Camada de IA / Modelo — nossa mudança principal

Aqui estava o pior acoplamento do projeto. O método `_process_frame` em `services/video_monitor.py` tinha **cerca de 45 linhas misturando seis responsabilidades**:

1. inferência YOLO;
2. filtro por classes-alvo;
3. desenho das bounding boxes (mutando o frame in-place);
4. state machine de detecções consecutivas;
5. regra de cooldown e emissão de eventos;
6. persistência.

Isso era exatamente o que a atividade pergunta: *"a chamada ao modelo de IA/YOLO está separada da regra de negócio?"*. A resposta honesta era **não**.

O que fizemos:

- Criamos `services/domain.py` com os value objects `Detection` e `AlertEvent` e quatro `Protocol`s (`Detector`, `AlertGate`, `EventStore`, `LlmClient`), todos com `@runtime_checkable`.
- Extraímos `YoloDetector` (em `services/vision/`) que cuida só de inferência e anotação. Crítico: `annotate()` retorna **cópia** do frame, nunca muta o original — resolve uma janela de race que existia antes.
- Extraímos `AlertEngine` (em `services/alerts/`) que cuida da state machine, do cooldown, da gravação do JPEG e da emissão de `AlertEvent`. Ele tem `threading.Lock` interno desde o nascimento, como precaução.
- Reduzimos `_process_frame` a seis linhas de orquestração:

```
def _process_frame(self, frame):
    detections = self.detector.detect(frame)
    annotated = self.detector.annotate(frame, detections)
    for event in self.alert_gate.submit(detections, annotated):
        self.event_store.save(event)
    with self.last_frame_lock:
        self.last_frame = annotated
```

- O construtor do `VideoMonitor` agora valida estruturalmente cada dependência com `assert isinstance(...)` contra o Protocol — quebras de contrato aparecem no boot, não em produção.

Por que melhorou: cada classe pode ser desenvolvida e testada isoladamente. Trocar de modelo de visão vira escrever um novo adapter. O risco de race no frame foi eliminado. E, principalmente, agora a resposta à pergunta obrigatória 4 da atividade é claramente sim.

1.8 Camada de integração externa

A única integração externa que existe hoje é o `OllamaClient`. Mesmo rodando em localhost, ele se comunica por HTTP, tem timeout configurável e implementa o port `LlmClient` — então funciona como adapter genuíno. A pasta `services/external/` foi criada vazia para receber o scraper da Parte 4.

1.9 Camada de web scraping

Inexistente em código, mas **estruturalmente reservada**. Decidimos antecipadamente que ela será **serviço separado** em `services/external/`, cumprindo um novo port em `services/domain.py`. Em nenhum momento haverá `requests.get( ... )` ou `BeautifulSoup( ... )` dentro de uma rota ou template.

1.10 Respostas às cinco perguntas obrigatórias

1. **A interface tem regra de negócio indevida?** Não. O `dashboard.js` só faz polling, montagem de DOM e gerencia o histórico em memória.
2. **O backend concentra a lógica principal?** Sim. `app.py` é fino; toda inteligência vive nas camadas de aplicação e infraestrutura.
3. **O acesso ao banco está isolado?** Sim. Uma busca por `sqlite3` no projeto inteiro retorna apenas `services/persistence/event_repository.py`.
4. **A chamada ao modelo de IA/YOLO está separada da regra de negócio? Sim, após o refator.** `YoloDetector` cuida do modelo; `AlertEngine` cuida da regra; `VideoMonitor` orquestra os dois via ports.
5. **A camada de scraping será serviço separado?** Sim. O lugar arquitetural (`services/external/`) já está reservado; o adapter implementará um novo port.

1.11 Estrutura final pós-refator


```

agrovision_ia/
├─ app.py
├─ services/
│   ├─ domain.py
│   ├─ config.py
│   ├─ video_monitor.py
│   ├─ vision/      yolo_detector.py, capture_store.py
│   ├─ alerts/      alert_engine.py
│   ├─ persistence/ event_repository.py
│   ├─ agent/        monitoring_agent.py, ollama_client.py, schemas.py
│   ├─ security/     (Parte 2)
│   └─ external/     (Parte 4)
├─ templates/index.html
├─ static/{dashboard.css, dashboard.js, captures/}
├─ docs/relatorio_atividade.md
├─ requirements.txt, .env, .env.example, .gitignore
└─ README.md

```

1.12 Métricas do refator

- `_process_frame` : de **46 linhas (6 responsabilidades)** para **6 linhas (uma: orquestrar)**.
- `services/` flat: de **8 arquivos** para **3 arquivos relevantes na raiz + 5 subpacotes** (o subpacote `security/` seria adicionado depois na Parte 2, fechando em 6).
- **4 Protocols** introduzidos nesta fase em `services/domain.py` (`Detector` , `AlertGate` , `EventStore` , `LlmClient`), todos validados por `isinstance` no boot. Um quinto port (`RateLimiter`) chegaria na Parte 2.
- **2 value objects** (`Detection` , `AlertEvent`).
- Comportamento funcional: **idêntico** ao anterior — mesmos eventos, mesmo JSON, mesma resposta do agente (validado manualmente subindo o servidor).

Parte 2 — Revisão de Segurança

Para a segurança, nosso passo inicial foi escrever o **modelo de ameaça**: contra quem estamos protegendo o sistema, e do que. Sem isso, qualquer mitigação vira teatro. Decidimos que o ativo principal é o **host** (CPU, memória, GPU do Ollama) e os dados operacionais; o atacante típico é um cliente HTTP não autorizado na rede; e ataques físicos ou comprometimento do filesystem do host estão **fora do escopo** (porque quem tem isso já comprometeu tudo).

A partir daí, classificamos cada achado em três categorias: **corrigido**, **conhecido e aceito** ou **ponto forte confirmado**.

2.1 Achados que decidimos corrigir

Achado	Onde	Mitigação
<code>/chat/stream</code> vazava <code>[ERRO]</code> <code>{exception}</code> ao browser	<code>app.py</code>	Mensagem genérica + <code>logger.exception</code> no servidor
<code>OllamaClient</code> propagava o <code>detail</code> bruto do Ollama	<code>services/agent/ollama_client.py</code>	Mensagem genérica em log + exception de domínio (<code>LlmUnavailableError</code>)
Sem rate limiting em <code>/chat/stream</code>	<code>app.py</code>	<code>InMemoryRateLimiter</code> (janela deslizante por IP)
Pergunta do usuário injetada direto no LLM	<code>services/agent/monitoring_agent.py</code>	<code>sanitize_user_prompt</code> + <code>DEFENSIVE_SYSTEM_RULES</code> no system prompt
Cabeçalhos HTTP de segurança ausentes	Todas as respostas	<code>SecurityHeadersMiddleware</code> (4 cabeçalhos)
Rotas operacionais totalmente abertas	<code>app.py</code>	Autenticação opcional via <code>X-API-Key</code> (opt-in por <code>.env</code>)

2.2 Achados conhecidos e aceitos (com justificativa)

Estes achados nós **encontramos**, mas decidimos **não corrigir** — cada um com motivo registrado:

- **Imagens em `/static/captures/` sem ACL.** A câmera é stream público oficial (Caltrans); não há identificação pessoal. O risco residual seria inferência por padrões temporais, que está fora do escopo.
- **SSRF teórico via `CAMERA_SOURCE` no `.env`.** Quem consegue editar o `.env` já comprometeu o host; mitigação adicional seria falsa segurança.
- **CORS não configurado.** Same-origin atende o uso atual; abrir CORS aumentaria a superfície sem benefício.
- **API key é mecanismo simples (não OAuth/JWT).** Coerente com o escopo didático. Anotamos como melhoria futura.

2.3 Pontos fortes que já existiam

Coisas que validamos como já corretas no projeto inicial:

- `.env` e `*.db` no `.gitignore` (sem credenciais versionadas).
- SQL **100% parametrizado** — zero risco de injection (confirmado com grep).
- Pydantic já validava `ChatRequest.question` (`max_length=4000`) e roles do histórico.
- `normalize_history` rejeitava roles forjadas (filtragem defensiva).
- Sem endpoint de upload — a pasta `uploads/` é dead folder.

2.4 Mitigações implementadas

Aplicamos o mesmo formato da Parte 1 — **antes/problema/solução/depois/por que melhorou** — para cada mitigação, mas em versão mais condensada.

Tratamento seguro de erros. Antes, qualquer falha do upstream Ollama (URL ofensiva, timeout, modelo inexistente) ia direto pro browser via `yield f"\n[ERROR] {error}"`. Agora, mensagens públicas vivem em `services/security/errors.py` (constantes como `GENERIC_AGENT_ERROR`, `GENERIC_UPSTREAM_ERROR`); o cliente recebe a string fixa, o operador recebe o traceback completo via `logger.exception`. Eliminamos um vetor clássico de *information disclosure* sem perder observabilidade.

Rate limiting em `/chat/stream`. Endpoint que dispara inferência LLM precisa de cap — sem isso, qualquer loop programático consome 100% da GPU/CPU. Implementamos `InMemoryRateLimiter` em `services/security/rate_limiter.py`, com janela deslizante por IP, `threading.Lock` e limites configuráveis no `.env` (`CHAT_RATE_LIMIT_PER_MINUTE=20`, `CHAT_RATE_LIMIT_WINDOW_SECONDS=60`). Quando excede, devolve HTTP 429 com mensagem amigável. Optamos por implementação manual (sem `slowapi` /Redis) porque o escopo é single-process e a auditoria fica simples: o arquivo inteiro tem 37 linhas.

Defesa contra prompt injection. Combinamos duas camadas: `sanitize_user_prompt` remove markers conhecidos (`<im_start>` , `[INST]` , `[SYSTEM]` , etc.) e normaliza espaços; e `DEFENSIVE_SYSTEM_RULES` é anexado ao system prompt instruindo o modelo a tratar a pergunta do usuário como **dado, não como comando**. A sanitização também é aplicada às mensagens com role `user` no histórico vindo do cliente — não confiamos no que o front manda. Nenhuma das duas é silver-bullet, mas filtram os ataques mais óbvios com custo desprezível.

Cabeçalhos HTTP de segurança. Criamos um middleware ASGI (`SecurityHeadersMiddleware`) que adiciona quatro headers em toda resposta: `X-Content-Type-Options: nosniff` , `X-Frame-Options: DENY` , `Referrer-Policy: no-referrer` e `Cache-Control: no-store` . Validamos via `curl -i` que os quatro estão presentes em `/health` . Custo: cerca de uma dúzia de linhas; benefício: imediato em qualquer browser moderno.

Autenticação opcional por API key. Esse foi o ponto mais discutido no grupo. Decidimos implementar **opt-in**: se a variável `API_KEY` no `.env` estiver vazia, o sistema roda em modo dev (rotas abertas, como antes); se estiver definida, todas as rotas operacionais (`/events` , `/agent/status` , `/chat/stream` , `/frame` , `/video_feed` , `/camera/status`) exigem o header `X-API-Key` . `/` (dashboard) e `/health` ficam públicos por design. A comparação usa `secrets.compare_digest` (tempo constante) para evitar timing attacks. No frontend, adicionamos um card "Credencial de Acesso" — visível só quando o servidor exige API key — que persiste a chave no `localStorage` e a anexa em todas as requisições. Como `` não permite header custom, trocamos por `fetch + Blob + URL.createObjectURL` .

Validamos empiricamente:

Teste	Resultado
<code>/health</code> (público)	200 com 4 cabeçalhos de segurança
<code>/events</code> modo dev sem header	200
<code>/events</code> modo seguro sem header	401
<code>/events</code> modo seguro com chave correta	200
<code>/events</code> modo seguro com chave errada	401

2.5 Respostas às cinco perguntas obrigatórias

1. **Senhas/tokens/chaves no código?** Não. Tudo em `.env` , que está no `.gitignore` . O `.env.example` versionado só documenta a estrutura.

2. **Rotas abertas sem validação?** Depende do modo. Em dev, sim — coerente com uso local. Com `API_KEY` definida, **todas as rotas operacionais exigem autenticação**.
3. **Dados do usuário são validados?** Sim, em camadas: Pydantic (`max_length=4000` , regex de role) → `normalize_history` (descarta roles forjadas) → `sanitize_user_prompt` (remove markers de injection) → rate limiter (impede flood antes mesmo de parsear).
4. **Risco de SQL injection, exposição, acesso indevido?** SQL injection: **não** (queries 100% parametrizadas). Exposição: mitigada por API key + `Cache-Control: no-store` . Acesso indevido: 401 com chave errada + comparação timing-constant.
5. **Mensagens de erro técnicas demais?** Já tratamos isso. Cliente sempre recebe string fixa de `services/security/errors.py` ; detalhes vão pro `logger.exception` em stderr.

2.6 Avaliação adicional (IA, scraping, upload)

A Parte 2 da atividade pede uma avaliação específica desses três pontos:

- **LLM (Ollama):** mitigado por sanitização + defensive rules + rate limit. Modelo roda localmente, sem leak pra terceiros.
- **YOLO:** o `.pt` vem do upstream oficial e está no `.gitignore` . Risco residual de pickle deserialization se o arquivo for substituído por malicioso — anotado como aceito.
- **Scraping (Parte 4):** será fonte externa não controlada. O membro responsável precisará tratar timeout, validar o JSON recebido e lidar com fonte fora do ar.
- **Upload:** não existe endpoint de upload. A pasta `uploads/` é dead folder mantida historicamente.

2.7 Estrutura da camada de segurança

```
services/security/
├─ __init__.py
├─ auth.py           build_api_key_dependency
├─ errors.py         mensagens publicas seguras
├─ headers.py        SecurityHeadersMiddleware
├─ prompt_safety.py  sanitize_user_prompt + DEFENSIVE_SYSTEM_RULES
└─ rate_limiter.py   InMemoryRateLimiter
```

Segue o mesmo padrão Hexagonal do resto do projeto: cada arquivo tem responsabilidade única, sem dependências cruzadas. O `app.py` é o único consumidor.

2.8 Limites e próximos passos

Para honestidade técnica, registramos o que **não** resolvemos:

- API key estática deveria virar OAuth/JWT em produção.
- Rate limiter in-memory funciona só single-process; multi-worker exigiria Redis.
- Capturas em `/static/captures/` continuam sem ACL — anotado como aceito.
- Não há audit log de quem acessou o quê.
- LLM local roda no mesmo processo do servidor; em produção, isolar em container.

2.9 Consistência arquitetural com a Parte 1

Depois de fechar as mitigações, fizemos uma **auditoria interna** comparando o estilo da Parte 2 com os padrões estabelecidos na Parte 1. Encontramos **três desvios reais** e conciliamos antes de seguir adiante. Achamos importante registrar isso porque mostra que arquitetura coerente exige disciplina contínua.

Desvio 1 — falta de Protocol para o rate limiter. Tínhamos definido na Parte 1 que toda fronteira de infraestrutura tem um port em `services/domain.py`. O `InMemoryRateLimiter` foi introduzido como classe concreta sem port. **Conciliação:** adicionamos `RateLimiter(Protocol)` em `domain.py`, declaramos `chat_rate_limiter: RateLimiter = InMemoryRateLimiter(...)` no `app.py` com `assert isinstance(...)`.

Desvio 2 — `monitoring_agent.py` perdeu a pureza de functional core. Para ativar a mitigação de prompt injection, importamos `sanitize_user_prompt` e `DEFENSIVE_SYSTEM_RULES` direto no módulo do agente. Tecnicamente continuava sem I/O, mas a propriedade "zero imports do projeto" foi quebrada. **Conciliação:** `build_agent_messages` agora **recebe** o sanitizer e as regras defensivas como parâmetros injetáveis (com defaults seguros). O `app.py` injeta as políticas concretas. `monitoring_agent.py` voltou a ser puro.

Desvio 3 — `OllamaClient` importava constante de `security/errors`. Acoplamento direto entre dois adapters de infraestrutura. **Conciliação:** criamos a exception de domínio `LlmUnavailableError` em `services/domain.py`. `OllamaClient` agora levanta essa exception (sem conhecer mensagens públicas); o `app.py` faz o mapeamento exception → mensagem pública na camada de transporte, como o Hexagonal prescreve.

Depois das três correções, validamos por smoke test que todos os Protocols continuam passando no `isinstance`, e que `monitoring_agent.py` e `ollama_client.py` não têm mais imports da camada `security/`.

Parte 3 — Melhoria de Código Gerado com IA

A atividade pede ao menos três trechos auxiliados por IA para revisão crítica. Como praticamente todo o projeto teve forte uso de IA na geração inicial, o trabalho aqui não foi procurar trechos "feitos por IA" (todos foram, em alguma medida), mas escolher casos onde a sugestão funciona em smoke test e **esconde** problemas estruturais relevantes — sintoma clássico do estilo "ship-it" que LLMs tendem a produzir.

Escolhemos **quatro trechos** em áreas diferentes do sistema.

3.1 Trecho A — `_load_env_file` : parser custom vs. `python-dotenv`

Antes. Tinha uma função custom em `services/config.py` que lia o `.env` linha a linha, dividia no primeiro `=` e fazia strip de aspas:

```
def _load_env_file(path: str = ".env") → None:
    ...
    for raw_line in file:
        ...
        key, value = line.split("=", 1)
        value = value.strip().strip('"').strip("'")
        if key and key not in os.environ:
            os.environ[key] = value
```

Problema. Quatro falhas concretas, todas reproduzíveis em `.env` reais:

- `VAR=valor # comentario` → o comentário entra no value.
- `export VAR=valor` → a chave fica `export VAR`.
- Escapes internos (`"texto com \"aspas\""`) quebram.
- Multilinhas (`VAR="linha1\nlinha2"`) não funcionam.

Estávamos reimplementando uma biblioteca padrão (`python-dotenv`) sem ganho.

O que fizemos. Adicionamos `python-dotenv==1.0.1` ao `requirements.txt`, removemos a função custom (13 linhas) e substituímos por `load_dotenv(override=False)` — o flag preserva o comportamento exato anterior (env já setada vence).

Por que melhorou. Robusto contra os quatro casos acima, padrão de mercado, 13 linhas a menos pra manter, custo zero (a lib é leve).

3.2 Trecho B — `video_feed` : busy-loop sem throttle nem disconnect handling

Antes. O gerador MJPEG em `app.py` emitia frames em loop, sem cap de FPS e sem tratar desconexão:

```
def frame_generator():
    boundary = b"--frame\r\nContent-Type: image/jpeg\r\n\r\n"
    while True:
        frame = video_monitor.get_frame_jpeg()
        if frame is not None:
            yield boundary + frame + b"\r\n"
```

Problema. `while True` sem `sleep` deixa a thread executora em 100% de CPU; sem cap de FPS, o cliente recebe tantos frames quanto a CPU emitir; quando o cliente desconecta, `GeneratorExit` é levantado dentro do `yield` e gera ruído nos logs do uvicorn.

O que fizemos. Adicionamos a variável `VIDEO_FEED_FPS` ao `AppConfig` (default 15, configurável no `.env`) e refizemos o gerador:

```
def frame_generator():
    try:
        while True:
            frame = video_monitor.get_frame_jpeg()
            if frame is not None:
                yield boundary + frame + b"\r\n"
            time.sleep(sleep_interval)
    except (BrokenPipeError, ConnectionResetError, GeneratorExit):
        return
```

Por que melhorou. A 15 FPS, cada iteração dorme ~67 ms; CPU dedicada ao MJPEG cai dramaticamente. FPS controlável sem mudar código. Disconnect tratado limpa — o generator retorna, recursos liberam.

3.3 Trecho C — `_process_stream` : lag de buffer HLS por `time.sleep` arbitrário

Antes. O loop de captura em `services/video_monitor.py` tinha `time.sleep(0.05)` entre `cap.read()` s, com a intenção (provavelmente sugerida pela IA) de "não saturar a CPU".

Problema. Em streams HLS, o `cv2.VideoCapture` mantém um buffer interno (no backend FFMPEG, geralmente 5-30 frames). Cada `cap.read()` puxa o **próximo da fila**, não o frame mais recente. Com `sleep(0.05)` entre reads e o stream emitindo a 15-30 FPS, frames acumulam mais rápido do que são consumidos: o sistema **vê o passado**. O lag cresce devagar — começa imperceptível e degrada com o tempo, exatamente o tipo de bug que escapa de smoke test.

O que fizemos. Duas mudanças combinadas:

```
try:
    cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
except cv2.error:
    pass
...
# loop sem time.sleep(0.05)
```

Por que melhorou. `BUFFERSIZE=1` instrui o backend a manter só o frame mais recente (best-effort, mas funciona no FFMPEG); remover o `sleep` elimina a janela de acumulação. O loop passa a ser limitado pelo tempo de inferência do YOLO (~50-100 ms em CPU), que é o ritmo natural do sistema. O lag crescente desapareceu.

3.4 Trecho D — Startup deprecated + warmup síncrono

Antes. A inicialização usava `@app.on_event("startup")` e chamava `ollama_client.warmup()` sincronamente:

```
@app.on_event("startup")
def startup_event():
    event_repository.init()
    video_monitor.start()
    ollama_client.warmup()
```

Problema. Dois ao mesmo tempo:

- `@app.on_event("startup")` está deprecated no FastAPI 0.93+.
- O `warmup()` faz chamada HTTP bloqueante; no primeiro boot, o Ollama precisa carregar 8 bilhões de parâmetros do `llama3` (~4.7 GB). Resultado: o servidor fica **15-30 segundos** sem aceitar requests. Healthchecks externos interpretam isso como serviço offline.

O que fizemos. Migramos para `lifespan` (asyncontextmanager) e movemos o warmup para uma thread executor com `asyncio.to_thread`:

```

@asynccontextmanager
async def lifespan(app: FastAPI):
    event_repository.init()
    video_monitor.start()
    asyncio.create_task(asyncio.to_thread(ollama_client.warmup))
    yield

app = FastAPI(title=config.app_title, lifespan=lifespan)

```

Por que melhorou. Sem DeprecationWarning. E o boot caiu de 15-30 s para **menos de 1 s** (medido empiricamente com `until curl --fail`). A primeira pergunta de chat pode ainda esperar o warmup terminar, mas isso é "espera ativa do usuário", não "tudo congelado".

3.5 Métricas e validação

Trecho	Antes	Depois
A — parser de <code>.env</code>	13 linhas custom	1 chamada de função (<code>load_dotenv</code>)
B — <code>/video_feed</code>	busy-loop, CPU 100%	sleep configurável (15 FPS default)
C — pipeline de visão	<code>time.sleep(0.05)</code> + buffer infinito	<code>BUFFERSIZE=1</code> + sem sleep artificial
D — boot do servidor	15-30 s (warmup bloqueante)	< 1 s

Cada refator foi confirmado por inspeção estática (verificando que os trechos antigos sumiram e os novos apareceram) e pelo boot real do servidor.

Parte 4 — Implementação de uma Camada de Web Scraping

Esta parte ficará a cargo de outro membro do grupo. O terreno arquitetural já está preparado: o subpacote `services/external/` foi criado vazio durante a Parte 1, e o padrão Hexagonal estabelecido permite que o futuro adapter entre sem refator adicional — basta criar um novo `Protocol` em `services/domain.py` representando a fonte externa, implementar o adapter em `services/external/`, e injetá-lo no composition root (`app.py`).

Conclusão

Encerramos as **três fases que ficaram sob nossa responsabilidade** com a sensação de que a entrega resolveu o que a atividade pediu sem inventar problemas extras. A Parte 4 (web scraping) será conduzida por outro membro do grupo, e o caminho já está preparado: o subpacote `services/external/` existe vazio, e o padrão Hexagonal que adotamos permite que o adapter de scraping entre como mais um, sem refator adicional.

O que ficou pronto

- **Parte 1 — Arquitetura.** Formalizamos a combinação Layered + Hexagonal + Pipeline. Extraímos `YoloDetector` e `AlertEngine`, reduzimos `_process_frame` de cerca de 45 para 6 linhas, reorganizamos `services/` em subpacotes coesos, e introduzimos os value objects e Protocols em `services/domain.py`.
- **Parte 2 — Segurança.** Criamos o pacote `services/security/` com cinco módulos coesos. Mitigamos seis achados (erros vazando, sem rate limit, prompt injection, cabeçalhos ausentes, rotas abertas, sem autenticação) e documentamos quatro como "conhecidos e aceitos" com justificativa. Depois auditamos a coerência com a Parte 1 e conciliamos três desvios — registramos isso na Seção 2.9.
- **Parte 3 — Refator de código auxiliado por IA.** Quatro trechos críticos foram trocados: parser custom de `.env`, busy-loop do `/video_feed`, lag de buffer HLS no pipeline de visão, e migração para `lifespan` com warmup não-bloqueante. O boot do servidor caiu de 15-30 s para menos de 1 s.

Aprendizados que tiramos do trabalho

1. **Funcionar não é o mesmo que estar bem feito.** Vários trechos auxiliados por IA passavam em smoke test e ainda assim tinham problemas estruturais (lag de buffer, busy-loop, parser frágil, warmup bloqueante). Sem revisão crítica, ficariam invisíveis até produção.
2. **A coerência arquitetural exige disciplina contínua.** A primeira versão da nossa Parte 2 quebrou levemente a pureza do functional core do agente (importando de `security/`). Só percebemos quando paramos para auditar. Isso virou a Seção 2.9 do relatório.
3. **Distinguir "corrigir" de "documentar e aceitar" é parte do trabalho.** Nem todo achado merece mitigação cosmética. Capturas sem ACL, SSRF teórico via `.env`, ausência de CORS — tudo isso foi conscientemente deixado como está, com justificativa.
4. **Métricas fecham o argumento.** "Melhorou" é vago; "boot de 15-30 s para <1 s" e "46 linhas para 6" são concretos. Onde conseguimos medir, medimos.

O que continuou aberto

- **Parte 4 (web scraping):** responsabilidade do outro membro do grupo.
- **Itens documentados como aceitos** ao longo das Partes 2 e 3 (API key estática, rate limiter single-process, capturas sem ACL, ausência de audit log) — todos com caminho de evolução registrado, caso o projeto saia do escopo didático.

Mensagem final

O AgroVision AI saiu desta revisão com **arquitetura mais defensável, superfície de segurança reduzida e trechos críticos auxiliados por IA submetidos a crivo humano**. A entrega não pretende ser exaustiva — nenhuma revisão real é —, mas é **honest**a: o que mudou tem justificativa, o que não mudou tem registro do porquê. Foi exatamente o que o enunciado pediu: tratar a IA como aceleradora, não como substituta, e usar o trabalho de desenvolvedor para transformar "funcional" em "tecnicamente sustentável".

Apêndice — Estrutura Final do Projeto

A árvore final consolidada será adicionada após o fechamento da Parte 4, para refletir o estado do projeto com o adapter de scraping em [services/external/](#) .